ONEDRAFT

CAD-AI — Aria Powered

Performance, Scalability & Capacity Architecture

Version 1.0

Document: OneDraft Performance, Scalability & Capacity Architecture Specification
System: OneDraft CAD-AI
Intelligence Layer: Aria
Document Class: Architectural & Implementation Stack Specification
Status: Architectural Specification
Predecessor: API Gateway, Rate Limiting & Service Protection Architecture Specification
Successor: Disaster Testing, Security Testing & Operational Readiness Architecture Specification

---

1. PURPOSE

The Performance, Scalability & Capacity Architecture establishes the engineering model governing OneDraft system performance, computational capacity, concurrent workloads, queue scaling, database growth, artifact processing, AI execution and production resource utilization.

The purpose of this subsystem is to ensure that OneDraft can support increasing numbers of organizations, users, projects, architectural models, geometry operations, validation workloads, documentation packages and AI interactions without compromising system correctness or tenant isolation.

Performance shall be treated as an architectural property rather than solely as an infrastructure optimization activity.

2. ARCHITECTURAL ROLE

The Performance, Scalability & Capacity subsystem provides the capacity-management layer connecting application execution, computational workloads, databases, queues, workers, object storage, APIs and AI services.

Primary responsibility: Define performance targets, capacity boundaries, scaling strategies and resource-management behavior.

Primary inputs: Workload volume, project complexity, concurrent users, geometry complexity, AI demand, queue depth, database load and infrastructure capacity.

Primary outputs: Capacity decisions, scaling actions, performance measurements, workload controls and resource allocation.

Authority: Performance controls shall protect system availability while domain services remain authoritative for project correctness.

3. PERFORMANCE MODEL

OneDraft performance shall be evaluated across multiple dimensions.

Latency: Time required to return a response or complete an operation.

Throughput: Quantity of operations processed within a defined period.

Concurrency: Number of simultaneously active operations.

Capacity: Maximum sustainable workload under defined service conditions.

Resource Utilization: Consumption of CPU, memory, storage, database connections, network bandwidth and computational resources.

Queue Depth: Number of pending asynchronous operations.

Scalability: Ability to increase capacity as workload increases.

4. PERFORMANCE CLASSES

OneDraft shall classify operations according to execution characteristics.

Interactive: Operations expected to provide rapid user feedback.

Standard: Ordinary API operations with moderate processing requirements.

Computational: Geometry, validation or model operations requiring substantial resources.

Document Generation: Drawing, rendering and package-generation operations.

AI: Aria and model-provider operations.

Background: Non-interactive maintenance or processing operations.

Bulk: High-volume operations affecting multiple resources.

Each class shall have appropriate timeout, concurrency and resource policies.

5. INTERACTIVE PERFORMANCE

Interactive operations shall prioritize predictable response latency.

Examples include:

Project Navigation

Requirement Editing

Model Metadata Queries

Drawing Inspection

Redline Entry

Review Actions

Aria Interaction Initiation

Large computational operations shall not block interactive application traffic unnecessarily.

6. ASYNCHRONOUS PERFORMANCE

Long-running operations shall execute through the Job & Worker Execution System.

Examples include:

Geometry Regeneration

Large-Scale Validation

Compliance Analysis

Drawing Generation

Rendering

CAD Export

BIM Export

Bulk Processing

The API shall return job identifiers where execution cannot reasonably complete within interactive response limits.

7. PERFORMANCE TARGETS

Performance targets shall be defined according to operation class.

Targets may include:

API Latency

Job Start Latency

Geometry Completion Time

Validation Completion Time

Drawing Generation Time

Export Completion Time

AI Response Time

Artifact Retrieval Time

Targets shall be measurable and versioned.

8. SERVICE-LEVEL OBJECTIVES

Critical OneDraft services shall define Service-Level Objectives.

Representative SLO dimensions include:

Availability

Latency

Error Rate

Job Completion

Queue Processing

Artifact Availability

Database Availability

SLOs shall be established for production environments and reviewed as workload characteristics evolve.

9. LATENCY BUDGETS

End-to-end requests shall use defined latency budgets.

A request budget may include:

```
CLIENT
 ↓
GATEWAY
 ↓
AUTHENTICATION
 ↓
APPLICATION
 ↓
DATABASE
 ↓
DOMAIN SERVICE
 ↓
RESPONSE
```

Each component shall avoid consuming disproportionate latency relative to the overall operation.

10. THROUGHPUT

System throughput shall be measured for major workload classes.

Metrics may include:

Requests Per Second

Jobs Per Minute

Projects Processed

Geometry Operations

Drawing Packages

AI Requests

Artifact Operations

Throughput measurements shall distinguish sustained capacity from short-term burst capacity.

11. CONCURRENCY

Concurrency limits shall protect shared resources.

Limits may apply to:

API Requests

Projects

Geometry Jobs

Validation Jobs

Drawing Jobs

AI Requests

Database Connections

Worker Tasks

Concurrency shall be managed at both organization and platform levels.

12. MULTI-TENANT CAPACITY

Capacity allocation shall preserve tenant isolation.

One organization shall not be permitted to consume all shared computational resources unless explicitly authorized by the applicable capacity policy.

Organization-level quotas shall integrate with subscription entitlements and platform resource limits.

13. FAIR RESOURCE ALLOCATION

The runtime shall support fair allocation of shared capacity.

Scheduling policies may consider:

Organization

Subscription

Priority

Queue Age

Workload Type

Resource Requirements

Concurrency Limits

The scheduling architecture shall prevent persistent starvation of ordinary workloads.

14. COMPUTATIONAL WORKLOADS

The primary computational workloads include:

Parametric Geometry

Model Regeneration

Constraint Evaluation

Compliance Computation

Validation

Coordination

Rendering

Documentation Generation

These workloads shall execute through scalable worker infrastructure.

15. GEOMETRY PERFORMANCE

The Geometry & Parametric Model Engine shall support performance-aware computation.

The system shall minimize unnecessary regeneration.

Where possible, computation shall use:

Incremental Updates

Dependency Graphs

Caching

Memoization

Spatial Indexes

Selective Recalculation

A change shall not require complete model regeneration where dependency analysis determines that only a subset of geometry is affected.

16. MODEL COMPLEXITY

Project Model complexity shall be measurable.

Complexity indicators may include:

Element Count

Space Count

Relationship Count

Constraint Count

Geometry Complexity

Assembly Count

Dependency Depth

Model Version Size

Complexity measurements may be used to predict computational requirements.

17. GEOMETRY WORKLOAD PARTITIONING

Large geometry workloads may be partitioned into independently executable operations where domain correctness permits.

Partitioning may occur by:

Level

Building Component

Spatial Region

Dependency Group

Processing Stage

Partitioned operations shall preserve deterministic model results.

18. QUEUE SCALING

The worker queue architecture shall scale according to workload demand.

Scaling indicators may include:

Queue Depth

Queue Age

Worker Utilization

Job Duration

Job Failure Rate

Pending Organization Work

Workers may scale horizontally when queue demand increases.

19. WORKER AUTOSCALING

Worker capacity may be automatically adjusted.

Representative worker pools may include:

General Workers

Geometry Workers

Validation Workers

Documentation Workers

Rendering Workers

Export Workers

AI Workers

Worker pools shall have independent resource policies where workload characteristics justify separation.

20. QUEUE PRIORITIZATION

Queues may support workload priority.

A representative priority model may be:

```
SYSTEM_CRITICAL
      ↓
INTERACTIVE
      ↓
STANDARD
      ↓
BACKGROUND
      ↓
BULK
```

Priority shall not override tenant security or authorization boundaries.

21. DATABASE PERFORMANCE

PostgreSQL shall remain the authoritative transactional persistence layer.

Performance architecture shall address:

Query Latency

Indexing

Connection Pooling

Transaction Duration

Lock Contention

Table Growth

Partitioning

Read Scaling

Maintenance Operations

Database optimization shall preserve transactional correctness.

22. DATABASE INDEXING

Indexes shall support common tenant-scoped access patterns.

Representative index dimensions include:

Organization ID

Project ID

User ID

Lifecycle State

Created Timestamp

Updated Timestamp

Revision

Job State

Indexes shall be evaluated against real query patterns rather than added indiscriminately.

23. CONNECTION POOLING

Database connection pools shall be controlled.

Pool configuration shall account for:

Application Instances

Worker Instances

Background Services

Administrative Services

The total connection footprint shall remain within PostgreSQL capacity.

24. READ SCALING

Read-heavy workloads may use read replicas where required.

Read replication shall not be used for operations requiring immediate transactional consistency unless the architecture explicitly guarantees the required consistency.

The primary database shall remain authoritative for writes.

25. DATABASE PARTITIONING

Large tables may use partitioning where workload volume justifies it.

Potential candidates include:

Usage Events

Lifecycle History

Audit Records

Telemetry Records

Event Records

Partitioning shall preserve tenant and temporal query efficiency.

26. DATABASE CACHING

Redis or equivalent caching infrastructure may reduce repeated database reads.

Caching may be used for:

Configuration

Entitlements

Session State

Frequently Accessed Metadata

Rate Limits

Computed Results

Cached data shall never become an unauthorized source of truth for transactional state.

27. CACHE INVALIDATION

Cached information shall have defined invalidation behavior.

Changes to authoritative records shall invalidate or update affected cached representations.

Security-sensitive cached data shall have appropriate expiration and invalidation controls.

28. OBJECT STORAGE SCALABILITY

Object storage shall support growth in:

CAD Files

BIM Files

PDFs

Images

Drawing Packages

Rendered Sheets

Model Artifacts

Exports

Object storage shall scale independently from transactional database capacity.

29. ARTIFACT DELIVERY

Large artifacts shall use direct or controlled object-storage delivery where appropriate.

The API shall avoid unnecessarily proxying large files through application servers.

Signed or authorized object references shall remain subject to tenant and permission controls.

30. NETWORK CAPACITY

Network capacity shall account for:

API Traffic

Artifact Transfers

CAD/BIM Imports

Exports

AI Provider Traffic

Internal Service Communication

Network bottlenecks shall be monitored independently from application latency.

31. AI PERFORMANCE

Aria workloads shall be treated as a distinct performance class.

AI performance shall consider:

Request Latency

Context Construction Time

Model Processing Time

Provider Latency

Token Throughput

Concurrent Requests

Retry Behavior

Provider Availability

AI provider latency shall not be allowed to block unrelated platform services.

32. MODEL PROVIDER SCALING

The AI provider architecture shall support provider abstraction.

Where configured and permitted, workloads may be routed between model providers according to:

Capability

Availability

Latency

Capacity

Cost

Policy

Model selection shall remain governed by the AI Intelligence architecture.

33. AI QUEUE MANAGEMENT

AI requests requiring asynchronous execution shall enter controlled queues.

AI workloads shall be isolated from geometry and documentation workloads where resource contention could affect system performance.

34. DOCUMENTATION PERFORMANCE

Drawing generation shall be scalable across independent jobs.

A drawing package may be decomposed into:

Plans

Elevations

Sections

Details

Schedules

Sheets

Rendering Tasks

Independent generation shall be coordinated through the documentation orchestration architecture.

35. EXPORT PERFORMANCE

Large CAD, BIM and document exports shall execute asynchronously where appropriate.

Export jobs shall expose:

Job State

Progress

Estimated Completion, where available.

Artifact Reference

Failure State

Export processing shall not block ordinary project interaction.

36. PROGRESS REPORTING

Long-running operations shall expose structured progress where technically meaningful.

Progress may include:

Queued

Initializing

Processing

Generating

Validating

Packaging

Completed

Failed

Progress values shall not imply precision that the underlying operation cannot support.

37. CAPACITY PLANNING

OneDraft shall maintain capacity models for major infrastructure components.

Capacity planning shall consider:

Organizations

Users

Projects

Model Complexity

Concurrent Jobs

Storage

API Traffic

AI Usage

Database Size

Capacity assumptions shall be documented and periodically reviewed.

38. CAPACITY TIERS

The infrastructure may define capacity tiers.

Representative tiers include:

Development

Test

Staging

Production

Production may itself contain scaling classes based on workload requirements.

Capacity tiers shall remain configurable through infrastructure management.

39. HORIZONTAL SCALING

Stateless application services shall support horizontal scaling.

Scaling may include additional:

API Instances

Worker Instances

Gateway Instances

AI Processing Workers

Documentation Workers

Horizontal scaling shall not introduce inconsistent tenant or project state.

40. VERTICAL SCALING

Services may be vertically scaled where workload characteristics require increased:

CPU

Memory

Storage I/O

Network Capacity

Vertical scaling shall be evaluated against horizontal scaling before becoming the sole capacity strategy.

41. ELASTIC SCALING

Where infrastructure supports it, OneDraft shall scale resources according to measured workload.

Scaling triggers may include:

CPU Utilization

Memory Utilization

Queue Depth

Queue Age

Request Rate

Latency

Database Load

Scaling policies shall include upper and lower bounds.

42. CAPACITY PROTECTION

The platform shall maintain reserved capacity for critical services.

Capacity protection may include:

Worker Reservations

Database Capacity Reservations

API Capacity

Emergency Scaling

Priority Queues

Critical system operations shall remain available during abnormal workload conditions where infrastructure capacity permits.

43. BACKPRESSURE

The system shall use backpressure when downstream capacity is constrained.

Backpressure may include:

Queueing

Rate Reduction

Request Rejection

Concurrency Reduction

Load Shedding

Retry Delays

Backpressure shall prevent uncontrolled resource exhaustion.

44. PERFORMANCE DEGRADATION

The platform shall define degraded operating modes.

Examples include:

Reduced Background Throughput

Reduced Rendering Concurrency

Delayed Bulk Processing

Restricted Large Jobs

Reduced AI Concurrency

Core project access shall remain prioritized where possible.

45. PERFORMANCE TESTING

Performance testing shall include:

Load Testing

Stress Testing

Spike Testing

Soak Testing

Concurrency Testing

Database Load Testing

Geometry Benchmarking

Drawing Generation Benchmarking

AI Latency Testing

Artifact Transfer Testing

Performance tests shall use representative architectural project models.

46. CAPACITY BENCHMARKS

OneDraft shall maintain benchmark datasets representing different project sizes.

Benchmark categories may include:

Small Residential

Medium Commercial

Large Commercial

Complex Institutional

Large Multi-Level

Benchmark models shall measure:

Model Load Time

Geometry Regeneration

Validation

Documentation Generation

Export

Storage

Database Impact

47. PERFORMANCE OBSERVABILITY

Performance telemetry shall integrate with the Observability architecture.

Metrics shall include:

Latency Percentiles

Throughput

Queue Depth

Queue Age

CPU

Memory

Database Load

Cache Hit Rate

Worker Utilization

AI Provider Latency

Artifact Transfer Rate

Performance telemetry shall support historical analysis.

48. PERFORMANCE & SECURITY

Performance controls shall not weaken security boundaries.

Scaling shall preserve:

Tenant Isolation

Authorization

Authentication

Auditability

Data Integrity

Artifact Security

AI Context Isolation

Caching and asynchronous processing shall preserve the same authorization model as synchronous operations.

49. ARCHITECTURAL INVARIANTS

The following invariants shall govern the Performance, Scalability & Capacity Architecture.

Predictable Performance: Major workloads shall have measurable performance characteristics.

Horizontal Scalability: Stateless services shall support horizontal expansion where practical.

Asynchronous Execution: Long-running operations shall not unnecessarily block interactive requests.

Tenant Fairness: Shared capacity shall be managed without uncontrolled tenant interference.

Backpressure: Capacity exhaustion shall result in controlled degradation rather than uncontrolled failure.

Database Integrity: Performance optimization shall not compromise transactional correctness.

Cache Safety: Cached data shall not bypass authorization or become an uncontrolled transactional source of truth.

AI Isolation: AI workload saturation shall not prevent unrelated core platform operations.

Observability: Capacity and performance behavior shall be measurable.

Determinism: Computational scaling shall preserve deterministic project results.

Security Preservation: Scaling mechanisms shall preserve tenant, authorization and data boundaries.

50. COMPLETION STATUS

The Performance, Scalability & Capacity Architecture establishes the performance and growth model for OneDraft.

The subsystem defines performance classes, latency targets, throughput, concurrency, computational workload management, geometry optimization, queue scaling, worker autoscaling, database performance, caching, object storage, network capacity, AI performance, documentation and export scaling, capacity planning, horizontal and vertical scaling, elastic scaling, capacity protection, backpressure, degraded operating modes, benchmarking, performance testing and operational observability.

The architecture establishes the capacity-management foundation required for OneDraft to scale from individual projects to large multi-tenant workloads while preserving project correctness, security, deterministic computation and service availability.

The next architectural layer establishes Disaster Testing, Security Testing & Operational Readiness, defining how the complete OneDraft platform is subjected to controlled failure, recovery, security and production-readiness testing.

Status: ARCHITECTURALLY DEFINED

Next Document: Disaster Testing, Security Testing & Operational Readiness Architecture Specification

Overall Architecture: OneDraft End-to-End System Integration